



LAB REPORT

Restaurant Management System

Object Oriented Programming Project

Submitted To	Mr. Nadeem
Member 01	Fiza Haroon (FA25-BSE-023)
Member 02	Awais Ahmed (FA25-BSE-021)
Department	Software Engineering
University	COMSATS University Islamabad

Table of Contents

1. Executive Summary
2. Introduction
3. Objectives
4. Tools and Technologies
5. System Modules
6. Class Details and Major Methods
7. OOP Concepts Implemented
8. Exception Handling
9. File Handling
10. GUI and Backend Connection
11. Working Flow
12. UML Diagram
13. Things Done in the Code
14. Future Enhancements
15. Conclusion

1. Executive Summary

The Restaurant Management System is a desktop-based Java application developed using JavaFX and Object-Oriented Programming principles. The system supports restaurant table reservation, customer data storage, food ordering, bill calculation, payment checking, and record saving using file handling. The project demonstrates a clear separation between the graphical user interface and backend logic, making the application easier to understand, maintain, and extend.

2. Introduction

Restaurants need a simple way to manage customers, table reservations, orders, payments, and saved records. This project provides these core functions through a JavaFX interface. The user interacts with forms and buttons, while backend classes such as Restaurant, Customer, Reservation, Order, Menu, and RestaurantFileHandler perform the main processing.

3. Objectives

- Create a JavaFX-based desktop application for restaurant operations.
- Store customer details such as name, phone number, and email.
- Reserve tables by checking table availability and guest capacity.
- Allow customers to select food items and create orders.
- Calculate total bill and handle payment validation.
- Save customer, reservation, and order records permanently in text files.
- Demonstrate major OOP concepts including inheritance, encapsulation, aggregation, constructors, static variables, ArrayList, exception handling, and file handling.

4. Tools and Technologies

Technology / Tool	Purpose
Java	Core programming language used for backend logic.
JavaFX	Used to build the graphical user interface.
Object-Oriented Programming	Used for class-based design and project structure.
ArrayList	Used to store dynamic lists of customers, tables, orders, reservations, and menu items.
File Handling	Used to save and read project records permanently.
NetBeans / Java IDE	Used for writing, compiling, and running the project.

5. System Modules

Module	Work Done
Reservation Module	Takes customer name, phone, email, date, time, and table number. It creates customer and reservation records.
Order Module	Allows the user to select food items and quantity. It creates an order and adds selected food items into it.
Billing Module	Displays order details and total bill. It checks paid amount and selected payment method.
Records Module	Reads old data from files and displays it in a TextArea.
File Handling Module	Saves customers, reservations, and orders into separate text files.

GUI Module	Provides screens for main menu, reservation, food order, billing, reservations, and saved records.
------------	--

6. Class Details and Major Methods

User

Parent/base class for common user data.

Main attributes: id, nextID, name, phoneNumber

Major Method	Use
User(name, phoneNumber)	Initializes user data and assigns an automatic id.
getId()	Returns user id.
setName(), getName()	Updates and returns name.
setPhoneNumber(), getPhoneNumber()	Updates and returns phone number.
displayUserDetails()	Displays user data.

Customer

Child class of User. It represents the restaurant customer.

Main attributes: email

Major Method	Use
Customer(email, name, phoneNumber)	Calls User constructor and stores email.
setEmail(), getEmail()	Updates and returns email.
getCustomerDataForFile()	Returns customer data in file saving format.
displayUserDetails()	Displays user details with email.

Restauranttable

Represents restaurant table information.

Main attributes: tableNumber, capacity, isReserved

Major Method	Use
Restauranttable(tableNumber, capacity)	Creates table with table number and capacity.
reserveTable()	Marks table as reserved.
FreeTable()	Marks table as free.
isAvailable()	Checks if the table is available.
displayTableDetails()	Displays table details.

FoodItems

Represents one food item from the menu.

Main attributes: itemId, nextItemId, itemName, price, category

Major Method	Use
FoodItems(itemName, price, category)	Creates food item with automatic id.
getItemName()	Returns item name.

getPrice()	Returns item price.
displayFoodItemsDetails()	Displays food item details.

Menu

Stores menu food items and searches them.

Main attributes: foodItems

Major Method	Use
Menu()	Creates ArrayList and adds default items.
addFoodItem(item)	Adds a food item.
addDefaultItems()	Adds Burger, Pizza, Biryani, Fries, and Cold Drink.
displayMenu()	Displays all menu items.
searchFoodItemByName(itemName)	Finds item by name and returns it.

Reservation

Stores reservation data and links customer with table.

Main attributes: reservationId, nextReservationId, noOfGuests, reservationDate, reservationTime, customer, table

Major Method	Use
Reservation(noOfGuests, customer, table)	Creates reservation and reserves table.
getCustomer()	Returns customer object.
getTable()	Returns table object.
setNoOfGuests()	Updates guests after validation.
getReservationDataForFile()	Returns reservation data for file saving.
displayReservationDetails()	Displays reservation with customer and table details.

Order

Stores customer order and calculates bill.

Main attributes: orderId, nextOrderId, customer, table, orderedItems, orderDate, orderTime

Major Method	Use
Order(customer, table)	Creates order for selected customer and table.
addItem(item)	Adds selected food item into order.
isOrderEmpty()	Checks if no item is added.
calculateTotal()	Calculates total bill.
getOrderDataForFile()	Returns order data for file saving.
displayOrderDetails()	Displays order details.

RestaurantFileHandler

Separate class for file handling.

Main attributes: customerFile, reservationFile, orderFile

Major Method	Use
saveCustomer(customer)	Writes customer data in customers.txt.
saveReservation(reservation)	Writes reservation data in reservations.txt.
saveOrder(order)	Writes order data in orders.txt.
readCustomersFromFile()	Reads saved customers and returns ArrayList.
getCustomersTextFromFile()	Returns customer file text for GUI.
getReservationsTextFromFile()	Returns reservation file text for GUI.
getOrdersTextFromFile()	Returns order file text for GUI.

Restaurant

Main backend controller class. It connects all backend classes.

Main attributes: customers, tables, reservations, orders, menu, fileHandler

Major Method	Use
Restaurant()	Initializes lists, menu, and file handler.
addCustomer(customer)	Adds customer and saves it.
addTable(table)	Adds table in table list.
reserveTable(customer, tableNumber, numberOfGuests)	Searches table and creates reservation.
createOrder(customer, tableNumber)	Creates order for selected customer and table.
addFoodToOrder(order, itemName)	Searches item in menu and adds it to order.
saveOrder(order)	Saves non-empty order.

RestaurantManagmentSystemm

Main JavaFX GUI class. It handles user interaction.

Main attributes: mainScene, restaurant, currentCustomer, currentOrder, customerName, customerPhone, customerEmail, selectedTable, totalBill, orderText

Major Method	Use
start(stage)	Creates main screen and adds tables.
showReserveScene(stage)	Shows reservation form.
showOrderScene(stage)	Shows order food screen.
showBillScene(stage)	Shows bill and payment screen.
showReservationScene(stage)	Shows saved reservations.
showRecordsScene(stage)	Shows all saved records.
main(args)	Launches the JavaFX application.

7. OOP Concepts Implemented

OOP Concept	Used In	Explanation
Encapsulation	User, Customer, Restauranttable, FoodItems, Order, Reservation	Data members are kept private and accessed through public methods.
Inheritance	Customer extends User	Customer reuses common user

		data and behavior from the User class.
Aggregation	Reservation has Customer and Restauranttable; Order has Customer, Restauranttable, and FoodItems	Objects are linked together but can exist separately.
Constructors	All major classes	Constructors initialize objects when they are created.
Static Variables	User, FoodItems, Reservation, Order	Used for automatic id generation.
ArrayList	Restaurant, Menu, Order	Stores multiple records dynamically.
Exception Handling	GUI and backend classes	Handles invalid input and file errors.
File Handling	RestaurantFileHandler	Saves and reads records permanently.

8. Exception Handling

Exception handling is used to keep the system running smoothly. If the user enters wrong input, the program shows an error message instead of crashing.

- Empty fields are checked before reservation.
- Quantity must be numeric and greater than zero.
- Order cannot be confirmed if it is empty.
- Payment amount must be numeric and greater than zero.
- File reading and writing errors are caught using IOException.

9. File Handling

The project uses text files for permanent storage. BufferedWriter and FileWriter are used to save data. FileWriter is used in append mode, so old data is not removed. BufferedReader and FileReader are used to read the saved data later.

File	Purpose
customers.txt	Stores customer name, phone, and email.
reservations.txt	Stores reservation id, customer name, table number, guests, date, and time.
orders.txt	Stores order id, customer name, table, ordered items, bill, date, and time.

10. GUI and Backend Connection

The GUI is connected with the backend through the Restaurant object. The GUI does not directly save files or calculate the main logic. It takes input and calls methods of Restaurant class. Then Restaurant uses other backend classes to complete the operation.

GUI Event	Backend Work
Reserve button clicked	Customer object is created, addCustomer() is called, and reserveTable() creates reservation and saves it.
Order Food opened	createOrder() creates an Order object for current customer and selected table.

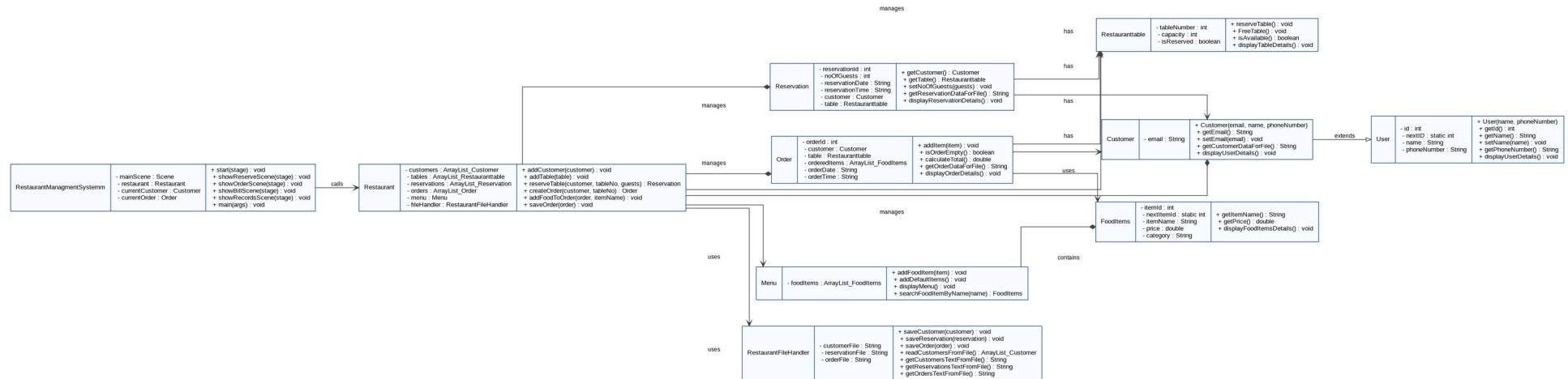
Add Item clicked	addFoodToOrder() searches food from Menu and adds it into Order.
Confirm Order clicked	saveOrder() saves order data in file.
View Reservations clicked	getReservationsTextFromFile() reads reservations.txt and shows data.
View Records clicked	All file text methods are called to show customers, reservations, and orders.

11. Working Flow

1. Application starts and five table objects are added in Restaurant class.
2. Main menu screen is shown with buttons.
3. User opens reservation screen and enters data.
4. GUI creates Customer object and sends it to backend.
5. Restaurant class creates Reservation and saves customer/reservation in files.
6. User opens order screen and creates an Order object.
7. Food is selected from ComboBox and added into Order.
8. Order class calculates total bill.
9. Confirm Order saves the order into orders.txt.
10. Bill screen checks payment and displays change or remaining amount.
11. Records screens read data from files and show it in TextArea.

12. UML Diagram

The UML diagram below shows the main classes, important attributes/methods, and relationships between them. Customer extends User. Restaurant manages customers, tables, reservations, and orders. Reservation has Customer and Restauranttable. Order has Customer, Restauranttable, and FoodItems. Restaurant uses Menu and RestaurantFileHandler. The GUI calls Restaurant backend methods.



13. Details of Things Done in the Code

- JavaFX main window is created using Stage and Scene.
- Main menu has buttons for reservation, ordering, billing, reservations, and records.
- Reservation form takes name, phone, email, date, time, and table selection.
- Customer object is created from GUI input.
- Tables are created as Restauranttable objects and added to Restaurant.
- Reservation object links Customer and Restauranttable.
- Food order screen creates Order object for current customer.
- Food is selected using ComboBox and quantity is entered using TextField.
- Food item is searched from Menu and added into Order.
- Total bill is calculated using calculateTotal().
- Payment screen checks paid amount and selected payment method.
- customers.txt, reservations.txt, and orders.txt are used for saving data.
- Records screen reads old data from files and shows it in TextArea.
- try-catch blocks handle major errors like empty fields, invalid numbers, and file problems.
- The code demonstrates inheritance, encapsulation, aggregation, ArrayList, exception handling, GUI, and file handling.

14. Future Enhancements

- Add login system for admin and staff users.
- Use a database such as MySQL instead of text files.
- Add receipt printing or PDF bill generation.
- Add table availability dashboard with colors.
- Add search and filter options for old records.
- Improve GUI design with CSS styling.

15. Conclusion

This Restaurant Management System is a complete Java OOP lab project. It combines a JavaFX GUI with backend classes. The project performs table reservation, customer management, food ordering, bill calculation, payment checking, and file-based record saving. It also demonstrates important OOP concepts such as inheritance, encapsulation, aggregation, constructors, static variables, ArrayList, exception handling, and file handling.